Московский государственный университет
имени М. В. Ломоносова

Майстренко Марат Алексеевич

«Раскрытие проблемы многократного порождения и уничтожения нитей»

Москва, 2025

1.Постановка проблемы

В современных вычислительных системах с многоядерными процессорами параллельное программирование становится необходимым для эффективного использования вычислительных ресурсов. Одним из популярных инструментов для параллельного программирования является OpenMP, который позволяет легко распараллеливать код с помощью директив компилятора. Однако при использовании OpenMP могут возникать проблемы, связанные с накладными расходами на управление потоками.

Многократное порождение и уничтожение потоков в OpenMP может привести к значительным накладным расходам на управление потоками. Это особенно актуально в задачах, где параллельные секции выполняются множество раз (например, в циклах). Каждый раз, когда создается новый набор потоков, система тратит время на их инициализацию, а после завершения параллельной секции — на их уничтожение. Эти накладные расходы могут существенно снизить общую производительность программы, особенно если параллельные секции выполняются часто.

2.Пример неправильного кода

Рассмотрим задачу умножения матрицы на вектор, которая хорошо поддается параллелизации. Если параллельная секция (например, с использованием директивы #pragma omp parallel for) вызывается многократно в цикле, то на каждой итерации будут создаваться и уничтожаться потоки. Это приведет к тому, что значительная часть времени выполнения программы будет потрачена не на полезные вычисления, а на управление потоками.

```
#include <iostream>
#include <vector>
#include <omp.h>
#include <cstdlib>
#include <ctime>

using namespace std;

void matrixVectorMultiply(const vector<vector<double>>& matrix, const vector<double>& vector) {
    size_t rows = matrix.size();
    size_t cols = matrix[0].size();
    vector<double> result(rows, 0.0);

    #pragma omp parallel for
    for (size_t i = 0; i < rows; ++i)
        for (size_t j = 0; j < cols; ++j)
            result[i] += matrix[i][j] * vector[j];
}

int main() {
    const size_t size = 1000; // Размер матрицы и вектора
    const int iterations = 100; // Количество итераций умножения

    vector<vector<double>> matrix(size, vector<double>(size));
    vector<double> vec(size);

    srand(42);
    for (size_t i = 0; i < size; ++i) {
        for (size_t j = 0; j < size; ++j)
            matrix[i][j] = static_cast<double>(rand()) / RAND_MAX;
        vec[i] = static_cast<double>(rand()) / RAND_MAX;
    }

    for (int i = 0; i < iterations; ++i)
        vector<double> result = matrixVectorMultiply(matrix, vec);

    return 0;
}
```

Основная проблема — **многократное создание и уничтожение потоков** в OpenMP. В коде директива #pragma omp parallel for находится внутри функции matrixVectorMultiply, которая вызывается в цикле iterations (100 раз). Следовательно, что на каждой итерации:

1. Создаются новые потоки.
2. После завершения функции потоки уничтожаются.

Последствия:

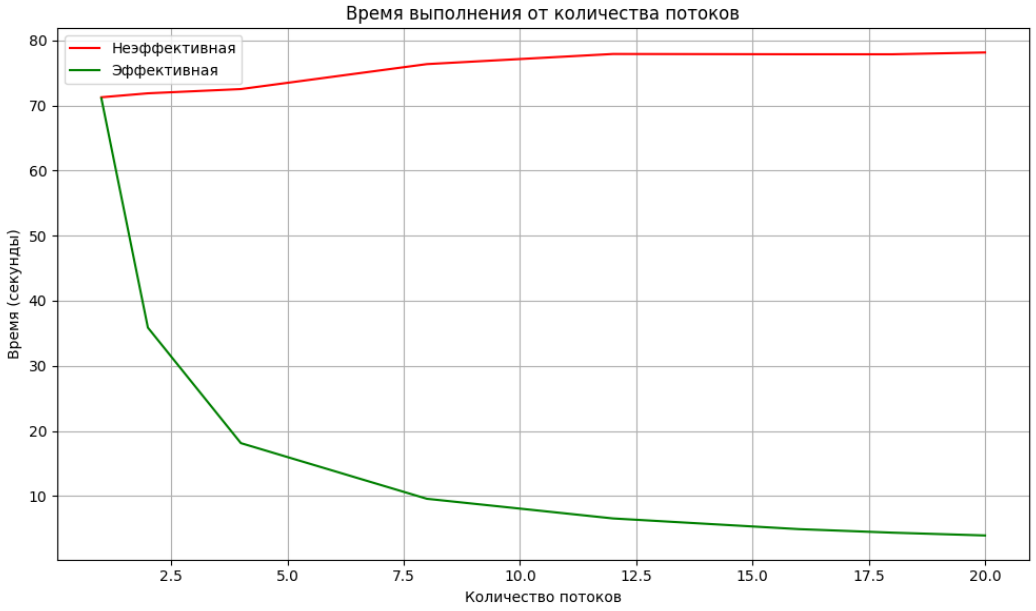
- **Накладные расходы**: Каждое создание и уничтожение потоков требует времени и ресурсов.
- **Снижение производительности**: Значительная часть времени выполнения тратится на управление потоками, а не на полезные вычисления.
- **Неэффективное использование ресурсов**: Потоки создаются заново вместо повторного использования.

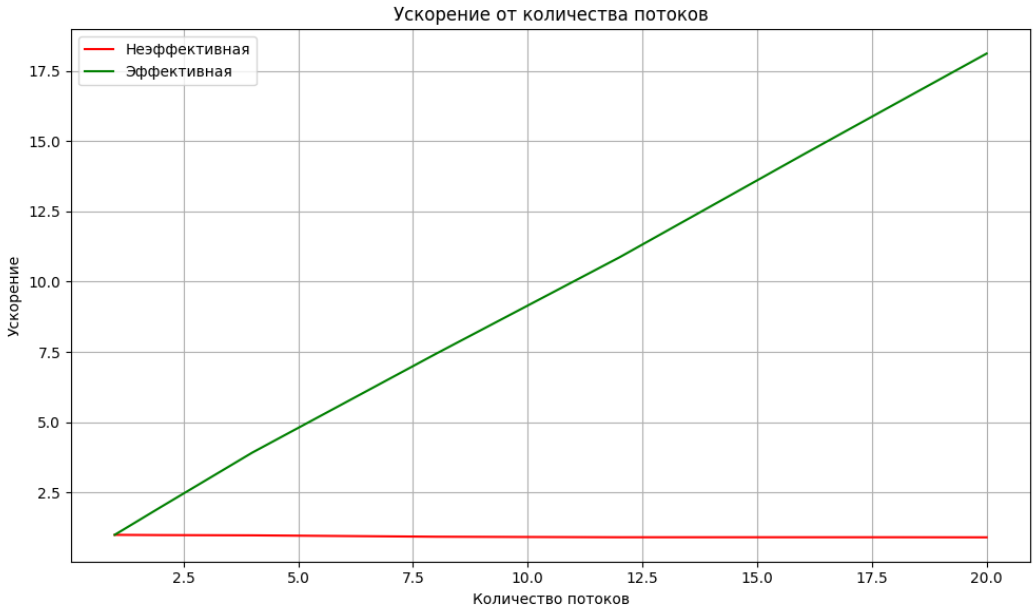
3. Исправление неправильного кода

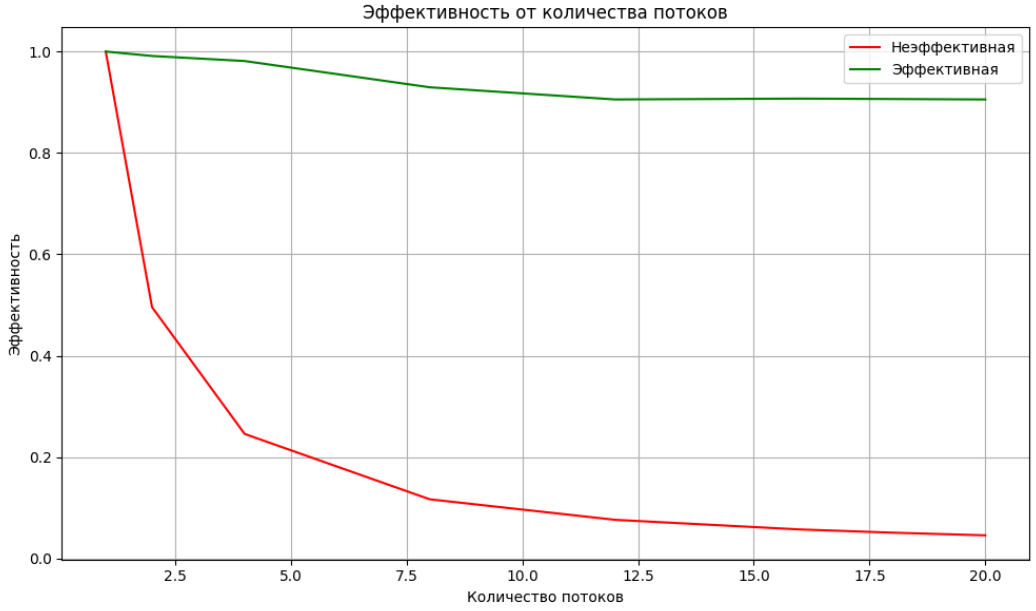
Чтобы устранить проблему, нужно **вынести параллельную секцию за пределы цикла**. Это позволит создать потоки один раз и использовать их для всех итераций

```
#include <iostream>
#include <vector>
#include <omp.h>
#include <cstdlib>

using namespace std;

void matrixVectorMultiply(const vector<vector<double>>& matrix, const vector<double>& vec) {
    size_t rows = matrix.size();
    size_t cols = matrix[0].size();
    vector<double> result(rows, 0.0);

    for (size_t i = 0; i < rows; i++)
        for (size_t j = 0; j < cols; j++)
            result[i] += matrix[i][j] * vec[j];
}

int main() {
    const size_t size = 1000; // Размер матрицы и вектора
    const int iterations = 100; // Количество итераций умножения

    vector<vector<double>> matrix(size, vector<double>(size));
    vector<double> vec(size);

    for (size_t i = 0; i < size; ++i) {
        for (size_t j = 0; j < size; ++j)
            matrix[i][j] = static_cast<double>(rand()) / RAND_MAX;
        vec[i] = static_cast<double>(rand()) / RAND_MAX;
    }

    #pragma omp parallel for // Потоки создаются один раз, итерации распределяются
    for (int i = 0; i < iterations; ++i)
        matrixVectorMultiply(matrix, vec);

    return 0;
}
```

4. Измерение ускорения и анализ результатов







Эффективная версия (потоки создаются один раз):

• Значительное улучшение производительности с увеличением числа потоков.

• Время выполнения уменьшается с 71.1237 секунд до 3.92724 секунд.

• Достигнуто ускорение примерно в 18 раз при использовании 20 потоков.

Неэффективная версия (потоки создаются каждую итерацию):

• Демонстрирует ухудшение производительности с увеличением числа потоков.

• Время выполнения увеличивается с 71.2747 секунд до 78.159 секунд.

• Отсутствие ускорения и незначительное замедление при увеличении числа потоков.

Замеры производились на вычислительном комплексе **IBM Polus.**

Количество потоков	**Эффективная версия**	**Неэффективная версия**
1	71.2211 сек	71.3532 сек
2	35.7589 сек	71.4338 сек
4	18.116 сек	72.6021 сек
8	9.53634 сек	76.4447 сек
12	6.53982 сек	78.0234 сек
16	4.90126 сек	78.0262 сек
18	4.4225 сек	77.9979 сек
20	4.02971 сек	78.4609 сек

5. Профилирование программы

Для анализа производительности программы был использован инструмент **perf**, который позволяет собирать детальную информацию о времени выполнения функций, использовании процессорных циклов и других метриках. Для сбора данных профилирования использовалась команда **perf record** с параметрами -F 1000 (частота сэмплирования 1000 Hz) и -g (сбор цепочки вызовов).

Анализ показал, что основное время выполнения (90.99%) приходится на функцию **matrixVectorMultiplyInefficient**, которая реализует неэффективное умножение матрицы на вектор, из которых только (44.47%) использовалось для вычисления. Также значительная часть времени (88.62%) была затрачена на создание и управление потоками, что связано с использованием OpenMP в неэффективной функции.

При этом функция **matrixVectorMultiplyEfficient**, отвечающая за эффективное умножение матрицы, занимает лишь 5.95% от общего времени. Это явно показывает, что основная проблема производительности программы связана с многократным уничтожением и созданием нитей.

```
Samples: 6M of event 'cycles:ppp', Event count (approx.): 25555479366586
  Children      Self  Command  Shared Object        Symbol
+   90.99%     0.00%  a.out    a.out                [.] main._omp_fn.2
+   90.99%     0.00%  a.out    a.out                [.] matrixVectorMultiplyInefficient
+   88.62%     0.00%  a.out    libc-2.17.so         [.] __clone
+   88.62%     0.00%  a.out    libpthread-2.17.so   [.] start_thread
+   88.59%     0.00%  a.out    libgomp.so.1.0.0     [.] 0xffffeffffd0d81c
+   44.54%    44.47%  a.out    a.out                [.] matrixVectorMultiplyInefficient
+   35.65%    35.63%  a.out    a.out                [.] std::vector<double, std::allocator<double> >::operator[]
+   12.69%     0.00%  a.out    [unknown]            [.] 0000000000000000
+   11.38%     0.00%  a.out    libc-2.17.so         [.] __libc_start_main
+   11.38%     0.00%  a.out    libc-2.17.so         [.] generic_start_main.isra.0
+   11.38%     0.00%  a.out    a.out                [.] main
+    9.44%     9.43%  a.out    a.out                [.] std::vector<double, std::allocator<double> >::operator[]
+    5.95%     5.95%  a.out    a.out                [.] std::vector<std::vector<double, std::allocator<double> >, std::allocator<std::vector<
+    4.40%     4.39%  a.out    a.out                [.] matrixVectorMultiplyEfficient
+    2.38%     0.00%  a.out    a.out                [.] matrixVectorMultiplyEfficient
+    2.22%     0.00%  a.out    [unknown]            [.] 0x00003ffff0367d20
```

6. Заключение

Исследование наглядно демонстрирует важность правильного использования потоков в параллельных вычислениях. Однократное создание потоков (эффективная версия) позволяет достичь значительного ускорения работы программы, в то время как многократное создание и уничтожение потоков (неэффективная версия) приводит к деградации производительности.

Эффективная версия программы показывает почти линейное ускорение до 18 раз при использовании 20 потоков, что говорит об оптимальном использовании вычислительных ресурсов. Это подчеркивает важность грамотного планирования и реализации параллельных алгоритмов.

Неэффективная версия наглядно демонстрирует, что накладные расходы на создание и уничтожение потоков могут значительно превышать выигрыш от параллельного выполнения задач. Это указывает на необходимость тщательного анализа и оптимизации